

1. Cadenas de caracteres en Python

◆ Qué son

Una **cadena de caracteres (string)** es una **secuencia de caracteres alfanuméricos** usada para representar texto.

Ejemplo:

```
nombre = "Camila"
mensaje = 'Hola, ' + nombre
print(mensaje) # Hola, Camila
```

Son fundamentales para procesar y mostrar información textual (nombres, direcciones, mensajes, etc.).

◆ Tipos de cadenas

◆ Tipos de cadenas

1. **Cadenas simples** Delimitadas por comillas simples ' o dobles ".
Ejemplo: `saludo = "Hola Mundo"`
2. **Cadenas multilínea** Usan triples comillas ' ' ' o " " " para texto de varias líneas.

```
texto = """Este es un
texto que ocupa
varias líneas."""
```

3. **Cadenas con caracteres especiales** Incluyen **secuencias de escape** como:

- `\n` → salto de línea
- `\t` → tabulación
- `\"` → comillas dentro del texto

```
mensaje = "Hola\nMundo"
print(mensaje)
# Hola
# Mundo
```

Operaciones y Métodos Comunes para el Manejo de Cadenas en Python

🌟 Operación / Método	📖 Descripción	⚙️ Funcionalidad principal	💡 Ejemplo en Python
Concatenación (<code>+</code>)	Une dos o más cadenas en una sola.	Combinar textos o resultados de manera clara.	<code>cadena1 + cadena2</code>
Slicing (<code>[:]</code>)	Extrae una parte específica de la cadena usando índices.	Obtener subcadenas o fragmentos de texto.	<code>cadena[0:5]</code>
Búsqueda (<code>find()</code> / <code>index()</code>)	Localiza la posición de una subcadena dentro de otra.	Validar datos o buscar coincidencias en texto.	<code>cadena.find("sub")</code>
Conversión (<code>upper()</code> , <code>lower()</code>)	Convierte todo el texto a mayúsculas o minúsculas.	Normalizar texto o comparar sin distinción de mayúsculas.	<code>cadena.upper()</code> / <code>cadena.lower()</code>
Eliminación de espacios (<code>strip()</code>)	Quita espacios en blanco al inicio y al final.	Limpiar entradas de usuario o datos importados.	<code>" hola ".strip()</code>
Reemplazo (<code>replace()</code>)	Sustituye una parte del texto por otra.	Corregir o transformar datos textuales.	<code>"casa".replace("a", "o")</code>
Comprobación (<code>in</code>)	Verifica si una subcadena existe dentro de otra.	Validar presencia de texto o condiciones.	<code>"hola" in cadena</code>

Formato de cadenas

Permite insertar valores dentro de un texto fácilmente.

✅ Con f-strings (forma moderna y recomendada)

```
nombre = "Camila"
edad = 20
print(f"Me llamo {nombre} y tengo {edad} años.")
```

📄 Me llamo Camila y tengo 20 años.

✅ Con `.format()`

```
print("Me llamo {} y tengo {} años.".format("Camila", 20))
```

✅ Con nombres de variables

```
print("Mi nombre es {n} y tengo {e} años.".format(n="Camila", e=20))
```

💡 El formato de cadenas sirve para combinar texto fijo y valores variables en un mismo mensaje.

Codificación y decodificación

Python almacena las cadenas internamente como texto (Unicode), pero los ordenadores y archivos trabajan con bytes, por eso es necesario codificar (texto → bytes) o decodificar (bytes → texto).

Unicode = el *conjunto* de caracteres

Unicode es **la lista universal de caracteres**: letras, símbolos, emojis, escritura árabe, chino, etc.

Cada carácter tiene un **código numérico** llamado *code point*.

Ejemplo:

- "A" → U+0041
- "ñ" → U+00F1

Unicode NO dice cómo guardar esos números en un archivo.

Solo define *qué caracteres existen* y *qué número tiene cada uno*.

UTF-8 = una *forma de guardar* Unicode en bytes

UTF-8 es una **codificación**, o sea, una manera concreta de convertir los números Unicode en **bytes**.

Por ejemplo, el carácter "ñ" tiene código Unicode **U+00F1**.

En UTF-8 se guarda como **dos bytes**:

ñ → C3 B1 (en hex)

Esos bytes son los que realmente van al archivo o por la red.

Codificar

Convierte texto (**str**) a bytes (**bytes**):

```
texto = "Hola mundo"
codificado = texto.encode("utf-8")
```

```
print(codificado)
# b'Hola mundo'
```

Decodificar

Convierte bytes a texto:

```
decodificado = codificado.decode("utf-8")
print(decodificado)
# Hola mundo
```

EJERCICIO 1

Escribe un programa que realice lo siguiente:

Pedir al usuario un texto o mensaje libre.

Ejemplo:

Hola, me llamo Ana y me gusta programar en Python.

1. Eliminar los espacios en blanco al inicio y al final usando `strip()`.
2. Preguntar al usuario si desea convertir el texto:
 - a mayúsculas (`upper()`)
 - a minúsculas (`lower()`)
 - o mantenerlo igual.
3. Preguntar si quiere reemplazar alguna palabra.
 - Si responde "sí", solicitar:
 - la palabra a cambiar,
 - y la nueva palabra por la que será sustituida (usa `replace()` o expresiones regulares).
 - Repetir esta opción tantas veces como el usuario quiera.
4. Concatenar una frase al principio del texto, por ejemplo:
"Mensaje final: " + texto.

5. Preguntar qué parte del texto quiere mostrar con slicing:

- desde qué posición empezar,
- y hasta qué posición terminar.
(Si no se introducen números, usar todo el texto).

6. Pedir una palabra o sílaba a buscar dentro del texto.

- Mostrar si aparece o no (**in**).
- Indicar su posición (**find()**).
- Mostrar un contexto (unos caracteres antes y después).

7. Mostrar un resumen final con:

- El texto limpio y modificado.
- La longitud total del texto.
- Las transformaciones realizadas.

BREVE INTRODUCCIÓN A FICHEROS

Tabla de instrucciones básicas para manejar ficheros en Python

Instrucción / Modo	Qué hace	Ejemplo	Resultado / Explicación	
<code>open("archivo.txt", "w")</code>	Abrir/crear fichero en modo escritura	<code>f = open("datos.txt", "w", encoding="utf-8")</code>	Crea o sobrescribe el archivo.	
<code>open("archivo.txt", "r")</code>	Abrir fichero en modo lectura	<code>f = open("datos.txt", "r", encoding="utf-8")</code>	Da error si el archivo no existe.	
<code>open("archivo.txt", "a")</code>	Abrir fichero en modo añadir	<code>f = open("datos.txt", "a", encoding="utf-8")</code>	Añade texto al final sin borrar lo anterior.	
<code>write("texto")</code>	Escribir texto en el fichero	<code>f.write("Hola\n")</code>	Añade el texto dentro del fichero.	
<code>read()</code>	Leer todo el contenido del fichero	<code>contenido = f.read()</code>	Devuelve una cadena con todo el contenido.	
<code>readline()</code>	Leer una sola línea	<code>línea = f.readline()</code>	Devuelve la siguiente línea del fichero.	
<code>readlines()</code>	Leer todas las líneas en lista	<code>líneas = f.readlines()</code>	Devuelve <code>["línea1\n", "línea2\n", ...]</code> .	
<code>close()</code>	Cerrar el fichero	<code>f.close()</code>	Importante para guardar cambios.	

EJERCICIO 2: Crear, guardar y leer un archivo de texto

EJERCICIO 3: El programa pedirá datos por teclado, los guardará en un archivo de texto y mostrará su contenido.